Structured Progress Port Specification

Package-wide observability contract for long-running mdstats operations

mdstats

Contents

1 Status

Implemented in `mdstats 0.19.69a0`.

Normative owner:

`mdstats/progress.py`

Current adopters:

`mdstats/plotting/atomic_density.py`
`mdstats/plotting/framework_density.py`
`mdstats/plotting/framework_dynamics.py`
`examples/plot_lta_mixed_alkali_density.py`
`examples/plot_na_lta_300k_all_species_density.py`

2 Purpose

Long-running scientific routines need observable progress without coupling numerical code to stdout, Python logging configuration, Plotly, notebooks, or one example script. The package therefore exposes a small structured progress port.

The separation is

computational module → `ProgressEvent` → `ProgressPort` → terminal, logger, GUI, notebook, or callback.

Computational modules emit events. Applications select how those events are rendered. The progress channel is observational only and does not influence scientific results, backend selection, resource admission, or numerical ordering.

3 Design goals

1. One package-wide event schema.
2. Silent operation by default.
3. Structured `X/Y` progress where a finite work count exists.
4. Stage-boundary messages for expensive indivisible operations.
5. No global logger configuration inside scientific modules.
6. No large arrays or mutable scientific objects retained by progress events.
7. A direct adapter to the standard `logging` module.
8. Backward compatibility with the former string callback.
9. A stable keyword-only port that future modules can adopt incrementally.
10. Negligible overhead relative to the reported work.

4 Non-goals

The progress port is not:

- a replacement for warnings or exceptions;
- a performance profiler;
- a telemetry upload service;
- a cancellation token;
- a resource-budget controller;
- a guarantee of uniform update intervals;
- a global logging configuration API.

Cancellation, tracing, and machine-wide telemetry require separate specifications.

5 Public data model

5.1 ProgressEvent

```python
@dataclass(frozen=True, slots=True)
class ProgressEvent:
    source: str
    stage: str
    message: str
    status: Literal[
        "started",
        "running",
        "completed",
        "warning",
        "info",
    ] = "running"
    current: int | None = None
    total: int | None = None
    unit: str | None = None
    metadata: Mapping[str, str | int | float | bool | None] = {}
    schema_version: str = "mdstats.progress-event.v1"
```

5.1.1 Field semantics

Field	Meaning
`source`	Stable dotted module or application identifier, for example `plotting.atomic_density`.
`stage`	Stable snake-case operation identifier, for example `field_realization`.
`message`	Concise human-readable state description.
`status`	Lifecycle state of the reported stage or work item.
`current`	Current work position when a finite total is known.
`total`	Total number of comparable work units.
`unit`	Human-readable unit such as `frames`, `fields`, or `shells`.
`metadata`	Small scalar diagnostic values only.
`schema_version`	Serialization contract identifier.

`current`, `total`, and `unit` are either all absent or form a consistent progress triple. `current` must satisfy

$$0 \leq \texttt{current} \leq \texttt{total}.$$

The event is immutable. Metadata is copied into a read-only mapping and accepts only JSON-like scalar values. Arrays, trajectories, meshes, and mutable planning objects are forbidden.

5.2 ProgressPort

```python
@runtime_checkable
class ProgressPort(Protocol):
    def emit(self, event: ProgressEvent, /) -> None:
        ...
```

This is the complete module-to-application interface. A custom consumer only needs an `emit()` method.

The use of a structural protocol follows Python's standardized protocol model specified by PEP 544. The mdstats event schema and lifecycle rules are package-local.

5.3 ProgressEmitter

A module binds one stable source to a port:

```
reporter = ProgressEmitter(
    progress_port,
    source="plotting.atomic_density",
)

reporter.started("field_realization", "resolving numerical plan")
reporter.update(
    "field_realization",
    "processing field",
    current=2,
    total=4,
    unit="fields",
)
reporter.completed("field_realization", "field complete")
```

The emitter validates every event before forwarding it. It contains no global state and performs no printing itself.

6 Built-in ports

6.1 NullProgressPort

The default no-op sink. Long-running functions remain silent when no port is supplied.

6.2 TextProgressPort

Writes human-readable progress to a caller-selected stream and adds elapsed time:

```
progress = TextProgressPort(
    label="LTA plot",
    stream=sys.stdout,
    show_source=False,
)
```

Example output:

```
[LTA plot |     43.0 s] field_realization [1/4 fields]: resolving samples and numerical plan
```

The elapsed-time origin belongs to the port, so wrapper and nested module events share one clock.

6.3 LoggingProgressPort

Forwards events to a caller-owned `logging.Logger` without configuring handlers, levels, or formatters:

```
progress = LoggingProgressPort(logging.getLogger("my-workflow"))
```

The human-readable event is the log message. The complete event dictionary is attached as `record.mdstats_progress`.

6.4 CallbackProgressPort

Adapts a structured callback:

```
progress = CallbackProgressPort(events.append)
```

This is useful for GUI queues, notebook widgets, tests, and external orchestration.

7 Standard module port

A computationally demanding public function should expose:

```
def expensive_operation(
    ...,
    *,
    progress: ProgressPortLike | None = None,
    progress_callback: Callable[[str], None] | None = None,
):
    ...
```

Rules:

1. `progress` is the normative keyword-only API.
2. `progress_callback` is a deprecated compatibility alias for the earlier string callback and may not be supplied together with `progress`.
3. The function resolves the port exactly once at its public boundary.
4. Nested modules receive the same resolved port, preserving one sink and one elapsed time origin.
5. Each module creates its own `ProgressEmitter` with a stable `source`.
6. A module never configures logging or decides how the caller displays events.

Resolution uses:

```
port = resolve_progress_port(
    progress,
    progress_callback=progress_callback,
    environment_variable="MDSTATS_PREPARE_PROGRESS",
    environment_label="mdstats-prepare",
)
```

The environment fallback preserves command-line diagnostics for callers that cannot modify an API invocation. An explicit port always takes precedence.

8 Reporting policy

8.1 Required events

A module should report progress when one or more of the following holds:

- the operation can plausibly exceed a few seconds;
- work scales with frames, atoms, fields, shells, graph states, or candidate objects;
- the operation contains a long preflight or planning stage;
- a large output is serialized;
- a worker process or expensive backend is launched.

At minimum, report:

1. stage start;
2. bounded-loop progress when a meaningful total exists;
3. stage completion;
4. a warning event when the module intentionally continues with a degraded mode.

8.2 Update frequency

Progress emission must not dominate runtime or flood terminals.

For a loop of length N, a reasonable default is approximately ten updates:

$$\Delta = \max\left(1, \left\lfloor \frac{N}{10} \right\rfloor\right).$$

Always emit the final update. Modules with very fast iterations should additionally avoid sub-second event storms. Inner Gaussian pairs, grid nodes, graph edges, and triangle operations are normally too fine-grained for direct reporting.

8.3 Stage names

Stage identifiers are stable machine-readable names. Messages may evolve without breaking consumers.

Recommended pattern:

```
input_validation
resource_resolution
frame_processing
backend_planning
field_realization
mesh_extraction
serialization
```

A stage name describes the operation, not the current prose message.

8.4 Metadata

Metadata should explain the current plan using small scalars, for example:

```
{
    "field_key": "atomic-density-0",
    "backend": "local_sparse",
    "sigma_angstrom": 0.0656,
    "thread_limit": 24,
}
```

Do not include arrays, per-frame records, mesh vertices, full paths, or arbitrary scientific objects.

9 Threading and process behavior

The default contract assumes coordinator-thread emission. A module may emit from workers only when its selected port is documented as thread-safe or events are first marshaled to the coordinator.

Current density mesh workers report completion from the parent process after each future resolves. Worker subprocesses do not write directly to the caller's port. This prevents interleaved terminal output and avoids serializing UI objects into workers.

Port failures are not silently swallowed. If a caller-provided port raises, the operation propagates that error because the caller's observability integration is invalid. Applications that require best-effort reporting should wrap their own port.

10 Current integration

10.1 Framework-dynamics preparation

Source:

`plotting.framework_dynamics.prepare`

Stages include:

- `scene_preparation`;
- `framework_registration` with frame counts;
- `trajectory_preparation`;
- `atomic_mean_graph`;
- `density_planning`;

- `density_realization`.

10.2 Atomic density

Source:

`plotting.atomic_density`

Each requested species or atom selection reports one `field_realization` item with backend, field key, and Gaussian width metadata.

10.3 Framework density

Source:

`plotting.framework_density`

Vertex occupancy and edge-length density are separate `framework_density_field` items.

10.4 Interactive rendering

Source:

`plotting.framework_dynamics.render`

Each requested highest-density-region shell reports one `density_mesh` item. Isolated worker completion is reported by the parent process with face count and elapsed time.

11 Future-module adoption checklist

Before opening a progress port in a new module:

1. Identify operations that dominate wall time.
2. Choose stable `source` and snake-case `stage` names.
3. Determine whether a real total exists; do not invent percentages.
4. Emit stage boundaries for indivisible work.
5. Emit coarse `X/Y` updates for bounded loops.
6. Keep metadata scalar and small.
7. Reuse a caller-resolved port in nested calls.
8. Add tests for event order, counts, final completion, and silent default behavior.
9. Preserve scientific behavior when the port is absent.
10. Document any worker-thread emission policy.

12 Compatibility

The former API accepted:

```
progress_callback: Callable[[str], None] | None
```

It remains operational through `LegacyTextCallbackPort` and raises a `DeprecationWarning`. Legacy callbacks receive formatted text without the source prefix. New code should use `progress=`.

No serialized scientific result or density field schema changes because progress records are transient observability events.

13 Validation requirements

Required tests include:

- event validation and immutability;

- scalar-only metadata;
- text, logging, callback, and null ports;
- legacy callback compatibility;
- explicit-port/environment precedence;
- mutual exclusion of `progress` and `progress_callback`;
- keyword-only `progress` on long-running public APIs;
- package examples using the shared port rather than a local reporter class;
- unchanged silent behavior when no port is supplied.

14 References

1. Python Software Foundation, PEP 544, "Protocols: Structural subtyping (static duck typing)," https://peps.python.org/pep-0544/.
2. Python Software Foundation, `logging` standard-library documentation, https://docs.python.org/3/library/logging.html.

The mdstats event schema, lifecycle rules, update policy, and adapters are original package design decisions rather than adaptations of an external scientific algorithm.